

SOFTWARE DESIGN 2

GROUP 32 AUTOVAULT GA2 AND GA7 REFLECTION

Group members

Member	Name	Student number
1	Mtshali Aphiwe	250858908
2	Mavhandu Khano	251056864
3	Maqasha Sanelisiwe Luyema	250237210
4	Nemahate Orinea	250799820
5	Thukhutha Ampfarisaho	250086387
6	Moswane Kelebogile	230467083

AutoVault: GA2 and GA7 Reflection

1. About this reflection

The reflection will consider the two graduate attributes covered by the Software Design 2 (SDN260S) unit through the current project, providing an analysis of how AutoVault meets them. This reflection is done for each graduate attribute, explaining what we did, why we did it, what went wrong and what we have learned from it. These descriptions are based on the graduate attributes provided in the course outline:

- GA2: Applying scientific and engineering knowledge. This generic attribute is cultivated by performing assigned exercises, assignments and projects which provide us with an opportunity to use the scientific and engineering knowledge obtained in the course to solve real-life problems. Knowledge about a concept is not sufficient; we have to be able to select an appropriate concept to solve a particular problem and apply it in a proper way.
- GA7: Sustainability and engineering activity. This generic attribute is acquired while considering the influence of software development on the sustainability and necessity to be economical with available resources. Software development is a specific type of engineering activity which is applied in numerous scientific and engineering domains; thus, its impact on sustainability may be significant. Small waste in one program may become substantial waste once this program will be executed many times on many computers.

Section 2 is related to GA2, Section 3 is related to GA7, while Section 4 is related to members' individual paragraphs. In every section, we have indicated evidence from our other project documents to enable verification of every claim made. The evidence is related to Final Project Report, User Guide and Testing Evidence document.

2. GA2: Application of Scientific and Engineering knowledge

2.1 The problem

The backups are forgotten or they delete their only copy of the file. Once it gets lost, damaged, or altered unintentionally, there is no option for retrieving the previous version. We were required to develop and implement an application that allows the user to select folders, backup the selected folders automatically, maintain history for a limited time, and retrieve any version from that history. This is what the given outline represents in terms of real-life scenario. This was not just a single programming assignment having one right solution but an entire software project that required the integration of concepts from different topics discussed in our course.

2.2 The knowledge we applied

Knowledge	Where we used it	Why it fitted
Algorithms	VersionSorter.mergeSort is a merge sort we wrote by hand, instead of using a library sort, so that we could apply what we learned about sorting. VersionSorter.binarySearch then searches the sorted list to find the newest version at or before a given time. Restore itself finds a version by its id.	Versions must be listed oldest first so that the user sees the history in order. Merge sort always takes O(n log n) time, even in the worst case, and it keeps versions with the same time in their original order. Binary search finds a version in O(log n) time because the list is already sorted, instead of checking every version one by one.
Data structures	BackupScheduler keeps ScheduledTask objects in a priority queue ordered by next run time. BackupManager keeps the backup jobs in a map, with the job id as the key.	The task that is due first is always at the front of the queue, so the scheduler only has to check one task each time it wakes up, not every task. A map finds a job directly by its id, so the program does not have to search through a list of jobs.
Recursion	RetentionPolicy.deleteRecursive deletes a version folder and everything inside it, including all of its subfolders and files.	Folders contain folders, which can contain more folders, so the data has a tree shape. A recursive method follows the shape of the data: it empties each subfolder before deleting it, and the base case is a plain file or an empty folder. Java cannot delete a folder that is not empty, so this order is needed.

Knowledge	Where we used it	Why it fitted
Hashing	HashDetector works out a SHA-256 hash of each file and compares the hashes to find the files that changed. The engine does not call it at present; BackupEngine.hasChanges compares file contents directly.	A hash shows that a file changed by its content, not by its date. If two hashes are different, the content is different, even when the date of the file has not changed. This makes the hash check more exact than a date check.
Object-oriented design	ChangeDetector is an interface with two implementations, TimestampDetector and HashDetector. BackupEngine does not call them at present; hasChanges compares file contents directly. BackupManager is a single entry point (a facade) that both the window and the console menu use to reach the rest of the system.	Because both detectors share one interface, another detector can be added without changing the others. Because both interfaces go through BackupManager, a change in the core does not need a change in both the window and the menu. This is the encapsulation and polymorphism we learned in the course, used for a real purpose.
Concurrency	The scheduler runs on its own thread, separate from the main program. The window runs backups and restores with SwingWorker, so the copying happens in the background and not on the thread that draws the window.	Automatic backups must not block the program while it waits for the next backup time, and the window must not freeze during a long backup or restore. Running the work on separate threads keeps the program responsive while it works.
File handling and storage	FileScanner lists the files in each chosen folder, BackupEngine copies them into a new version folder, and RestoreService copies a chosen version back. The jobs are saved to jobs.dat with Java serialization.	Backups are only useful if every file is copied correctly, so file copying had to be reliable. The jobs must also survive a restart, so the user does not have to set up the folders and times again each time the program opens. Serialization saves the job objects to a file and reads them back with little extra code.

2.3 An engineering decision

Choosing where we should save the versions gave us two alternatives. The first one involved saving only the modified files from each version. Such an approach would

take up less disk space; however, if we wanted to retrieve a certain version, we would also have to have access to the previous versions because otherwise the unchanged files would be missing. As for our policy of keeping versions, it involves deleting the oldest ones, which can create problems in the restoration process.

Our second option involved creating full copies of all versions. We picked full copies as well. In this way, it will be easier and safer to restore or retain any version without interfering with others since each of them can stand independently. 5 versions per each document is the maximum number allowed by us since it will help us to keep the disk space within acceptable limits. Our Plan 1 for Check-In stated that only modified files should be processed. It was our deviation from the plan, and it was done for safety reasons.

We still avoid any unnecessary work, as the engine determines whether there is anything new, and if not, there is no need for the back-up. This reflects our evaluation of the benefits and drawbacks of each alternative before we made the decision.

2.4 What went wrong and what we learned

Data should match disk: First, the retention policy deleted the version folders from the disk, but the job history would still refer to those versions, making the user able to select one of them, thus making restoration impossible due to non-existing version folder. We solved the problem by deleting the version from the job, if its folder was deleted, ensuring the consistency between saved data and disk and checking it (N6, N7, and the first defect in the defects table). We concluded that if the program stores the information about some item on the disk, any changes to the disk should result in changing that information.

Plan for failures: The initial failure of the backup caused the entire program to crash because we had accounted for only the scenario where everything worked. But in practice, the folder could be missing, the file could be locked, or the disk could be full. Our revised program is capable of handling failure of the backup and reporting it back to the user (and testing E10 and E15 for the second defect in the defects table), but we learned our lesson the hard way.

Understand how the framework works. In the window, when show() is called, it triggers setVisible, which calls show(), and so on, going into an infinite loop. Reading about how Swing calls the methods helped to realize why – setVisible calls show() within the framework, meaning our show() called itself in a loop. Now, the class calls super.show(). It was learned that when overriding a method from the framework, it is necessary to understand how the framework uses the method.

Fitting together the pieces: Everyone programmed their own classes separately, and they didn’t necessarily fit together with the UML and other classes. There were some missing methods, and there were some names and return types which did not correspond with the others. We added the missing methods in the classes without changing anything in the code written by each individual. We realized that in team projects, as much attention should be paid to the joints between the pieces as to the pieces themselves.

2.5 What we would do differently

We would agree the method names, parameters and return values of the classes earlier, and check them against the UML before anyone starts coding, so the parts fit the first time. We would also write the tests while building each class, not after it, so that the defects above would have been found sooner, when they were smaller and easier to fix. We would also join the parts together more often during the project, instead of near the end, so that problems between the parts show up early.

2.6 Evidence

- Final Project Report: Sections 4 to 6 (design, implementation, algorithms), which explain the design described in this section.
- Testing Evidence: tests N1 to N10, E1 to E15, W1 to W9, and L1 to L6, and the four defects in the defects section, which match the problems described in Section 2.4.

3. GA7: Sustainability and Impact of Engineering Activity

3.1 Why this matters for a backup tool

The back-up application will continually repeat itself, at regular intervals, without any human intervention. The inefficiencies in a single repetition will keep repeating themselves each time it repeats, and will also keep repeating themselves in each computer in which it is used. This is how a tiny application can end up having a significant impact on resources like disk space, processing power, energy consumption, and user time. It has been mentioned in our course outline that software development has become an integral part of a large variety of engineering processes, hence its significant impact on sustainability.

The backup solution is a perfect example of such technology. This is why we made sure that AutoVault would be sustainable in its resource utilization, looking into sustainability from different angles – including not just energy and hardware, but software life-cycle as well.

3.2 A simple example

Now suppose we have a folder with 1 GB size and we make backups every 30 minutes. This means we have 2 backups per hour and 24 hours a day, which equals 48 backups per day. If AutoVault backed up everything each time, it would result in writing 48 × 1 GB = 48 GB of data to the disk per day, despite the absence of any changes in files. Without any limitation on the number of versions, it will keep all these 48 GB backups every day, which is over 1.4 TB per month. In our system, the engine will check for changes and skip backup if there were none, thus doing nothing.

However, whenever there are modifications made to the files, just the last five versions are saved; thus, the amount of backup space required never exceeds 5 × 1 GB = 5 GB for this particular folder, no matter for how long the program is run. These figures may vary, but this shows the constraints of the system.

3.3 How we used resources efficiently

Resource	What we did	What is still wasteful
Disk space	RetentionPolicy keeps the newest 5 versions of each job and deletes the older version folders, so the backups cannot keep growing. Users can skip file types such as tmp and log, which are never copied because they are not worth keeping. No backup is made when nothing changed, so no space is used for a copy that is the same as the last one.	Every version is a full copy, so files that did not change are stored again in every version. Storing only the changes, or compressing the backups, would use less space, but it would make restore and retention more complex (see Section 2.3).
Processing time and energy	BackupEngine.hasChanges checks for changes before copying anything, so no copying is done when nothing changed. It first compares the number of files and folders, and only then compares file contents, stopping at the first difference. The scheduler sleeps between checks and only looks at the task at the front of its priority queue, so its work stays small even when there are many jobs.	The scheduler wakes up twice a second, even when the next backup is 30 minutes away, so it makes many checks that are not needed. It could sleep until the next task is due, which would use less processing time.
The user’s time and effort	Backups run by themselves at the interval the user sets, so no one has to remember to copy files by hand, and a lost file can be brought back quickly. A window and a menu, clear messages and a User Guide make the tool easy to learn and use, so the user spends less time on it.	The window starts automatic backups when a job is added, but it has no button to stop or restart them for an existing job.
The team’s time, and future developers	Four packages, one entry point (BackupManager), and a ChangeDetector interface keep the code organised and easy to change, because a change in one part does not spread to the others. The UML, comments, logging and tests make problems quicker to find and fix, so the software can be maintained for longer instead of being rewritten.	Some classes had to be adjusted to fit together, which took extra team time near the end. The UML also needs updating to match the final code, or it could mislead future developers.

Resource	What we did	What is still wasteful
The user’s data	Each version is a separate full copy, so a restore never depends on another version, and deleting an old version cannot damage a newer one. Errors are shown to the user and written to AutoVault.log, so a problem is not hidden and can be found and fixed.	Backups are not encrypted, so anyone with access to the backup folder can read the files. There is also no warning before a restore overwrites a file, so a user could replace a newer file by mistake.

3.4 What we learned

Efficiency became one of the design choices, and it was evident that adding it from the beginning was much easier than adding it afterward. The change check and the retention limit were minimal pieces of code, yet they took away a lot of redundant coding each time the utility was run. In case we would have made these decisions later on, we would have to modify multiple classes simultaneously. What’s more, we have realized that the choice which saves one kind of resources may lead to expense of the other one. Full copies are easy to restore but they require more disk space.

We had to determine what is most important for this tool, and it turned out to be the security of the user’s data, as a backup, which cannot be recovered, is worthless. This point was included into the Final Project Report so that anybody who will maintain the software in the future would know the reason for this choice. Last but not least, we have understood that sustainability in software development does not only relate to energy.

3.5 What we would improve

- Store only the modified files or compress the backup data to reduce storage requirements without compromising on restoring each version.
- Have the scheduler sleep until the next operation is required, rather than waking up twice every second, to conserve computational resources.
- Allow the user to set the number of backup versions kept per task, where critical directories keep more histories and others require less storage.
- Conduct automated backups in the background, where the application doesn't need to be in the foreground for the backups to be conducted.

3.6 Evidence

- Final Project Report: Section 6 (algorithms and data structures), Section 4 (system design) and Section 9 (limitations), which explain the choices and trade-offs described in this section.
- Testing Evidence: E12 (no backup when nothing changed) and N6 and N7 (old versions and their folders are deleted), which show that the efficiency features work as designed. Additionally, the exclusion of certain extensions is verified in the engine logic.
- User Guide: “What AutoVault Does”, “How Backup is Done” and “Tips and limits”, which explain these features and limits to the user in plain language.

4. Individual Paragraphs

Aphiwe Mtshali (Project Lead / Integration)

I am the project lead. I built BackupManager and Main, and I joined everyone's classes into one working system.

For GA2, object-oriented design was applied to create BackupManager that represents a single access point (facade), so neither the window nor the console menu has direct access to the engine, scheduler or file classes. A map was used to find a job by its id, serialization was used to save and load jobs in jobs.dat, and a background thread was used to perform automatic backups. Missing methods were added to other members' classes, but I did not rewrite their implementations. For GA7, BackupManager makes sure that changes have been made before a backup is performed, so no copy is made when the folders are identical, and it requests the retention policy to delete old copies after each backup, so that the disk usage will not increase indefinitely. The scheduler thread is created once the user initiates the automatic backups for the first time, and it is a daemon thread, which means it does not prevent the JVM from shutting down when only daemon threads remain. A daemon thread can still consume resources while it is idle, which is why the scheduler should sleep until the next backup is due. The drawback is that BackupManager is not synchronized, so in a future version all of its methods should be synchronized.

Khano Mavhandu (Core Backup)

I have implemented the backup class hierarchy: FileScanner, BackupEngine and RestoreService. FileScanner discovers the list of files in selected directory, BackupEngine duplicates them in the new version folder, and RestoreService duplicates selected version to the folder selected by the user.

In case of GA2, I have introduced the file handling and exception handling to ensure reliable copying and restoring of the files, and also introduced the check that the backup engine will refuse the folder for backup if it is a subfolder of the source folder, otherwise it would just back itself up. FileScanner is shared between the engine and restore service, so no need to write the same thing twice. In case of GA7, BackupEngine.hasChanges method determines if this is the first backup, if file was added or deleted, and if file was modified, and ignores the file types specified by the user as unneeded for backup like tmp and log. All versions are full copies, which occupy more space than storing just the differences, but the benefit is that all versions can be recovered independently, and the limit of 5 versions allows to keep space limited. The only way to improve this process is to compress copies.

Luyema Maqasha (Scheduling and Retention)

I created BackupScheduler, ScheduledTask and RetentionPolicy. For GA2, I implemented priority queue so that a task due for execution comes up first in the queue. ScheduledTask implements interface Comparable, and method compareTo sorts the tasks based on their next scheduled time. The scheduler works on its own thread, and all methods are synchronized so that the menu and the scheduler do not modify the queue simultaneously. The actual backup procedure takes place outside of

synchronized code so that a long running backup procedure does not prevent the user from scheduling and canceling jobs. For GA7, I used recursion in deleteRecursive since a version folder contains folders and Java can only delete an empty folder. RetentionPolicy retains only the last 5 versions so that disk space usage stops increasing, and the scheduler deals only with the job located at the head of the queue so that its operations remain efficient regardless of number of jobs. There is another waste in the scheduler that wakes up twice every second, although the next scheduled backup will only happen after half an hour. I would like to improve it by making the scheduler sleep until the next task arrives.

Orinea Nemahate (Change Detection and Algorithms)

I developed the classes: ChangeDetector, TimestampDetector, HashDetector and VersionSorter. ChangeDetector is an interface with two implementations, so another detector can be added without changing the others; at present the engine does not call them, because BackupEngine.hasChanges compares file contents directly. While TimestampDetector compares the date of files, HashDetector computes SHA-256 hash of the file thereby detecting a change based on the file contents rather than the file date. The class VersionSorter implements a customized merge sort to arrange all versions in increasing order of age. Its complexity is O(n log n) and it retains the order of versions with the same age as in the original list. I have implemented binary search for finding the latest version that occurs before the specified time within O(log n) instead of traversing all versions one by one. For GA7, TimestampDetector only reads the file date, while HashDetector must read all the bytes of the files, so it suits cases that need an exact answer. The hashes used by the HashDetector are kept in memory and discarded after shutting down the program.

Ampfarisaho Thukhutha (Model, Storage and UI)

The classes of BackupJob and BackupVersion, saving and loading of the jobs, as well as the two forms, that is, MainWindow and CliMenu have been developed by me. GA2 introduces validation of BackupJob and throws exception if some of its properties are invalid, namely: empty name of the job, interval lower than one minute, and backup directory being inside of the source directory. This is the correct use of encapsulation that makes the data safe. The jobs and their versions are the serialized classes, and they are saved into jobs.dat file. In the MainWindow I use swing events, the table being updated every five seconds and SwingWorker, so that the backups and restore process could run in the background, while the window will not be blocked. I have figured out that our show() method has to call super.show() because of swing setVisible invoking show(). Regarding GA7, the checking of the jobs prevents from starting them, because they will be useless – for instance, the backup directory inside the source directory would copy itself. The jobs are saved in the sense that the user does not have to configure everything each and every time; and the background workers mean that the user does not have to wait for the backup to complete. The only drawback is that the window has no button to stop or restart automatic backups for an existing job.

Kelebogile Moswane (QA, Logging and Documentation)

The tasks I was responsible for included BackupLogger and LogEntry besides Test Plan, UML maintenance and documentation. In GA2, BackupLogger records every event, including time, level, and message in AutoVault.log file and also loads the old entries during startup. Thus, even in case of restarting, the history will still be maintained. I have done tests for logger in both usual and extreme conditions; the logger was tested for correct logging and reading of events, complete information about every field, exportation and restarting abilities, as well as proper processing of not being able to write. Usual and extreme cases of testing strategy were used for the entire system in GA2 in Testing Evidence document. Moreover, I have maintained UML according to the code. In GA7, logging provides an opportunity to find the problem quickly, without any unnecessary effort from the developers' side and also testing helped to detect problems before the user does. This is where the user guide, Final Project Report, and UML come into play. They will allow people to reuse and customize the AutoVault on their own without needing our help in the future. This will ensure that the software is used for a long period of time. The only other step that is left is to test the log in cases of excess size.